

Nota: O exame deve ser respondido em 3 folhas de prova separadas:

- 1 Folha para responder às três perguntas Teóricas
- 1 Folha para responder à 1ª pergunta da Prática
- 1 Folha para responder à 2ª pergunta da Prática

Duração Total Exame (T + P) : 2h:00m + 30m tolerância

5 de Julho de 2024

Parte Teórica

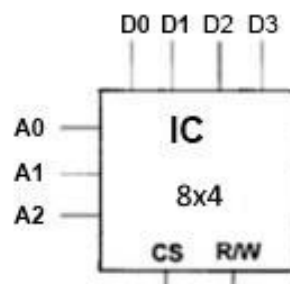
- 1** Um tipo mais comum de discos rígidos é o disco *Winchester*, composto por uma unidade selada com um conjunto de pratos sobrepostos, dentro de uma caixa de metal com uma pequena separação entre eles. Considere um disco deste tipo com apenas um prato de dupla face, 128 sectores, 1024 cilindros onde cada pista armazena 32KBytes por cada sector. **(2 Val)**

Calcule:

- a) Capacidade de armazenamento de cada cilindro.
- b) Capacidade de armazenamento total do disco.

- 2** Considere o circuito integrado de memória RAM da figura, onde A_1, A_0 representam linhas de endereço, $D_7, D_6, D_5, D_4, D_3, D_2, D_1, D_0$ representam linhas de dados, R/W representa a linha de leitura/escrita e CS a linha de *Chip Select*.

Pretende-se dimensionar uma memória capaz de armazenar 13 bytes. Faça um esboço, associando um número mínimo de circuitos integrados do tipo IC1, de forma a obter a memória RAM pretendida. Deverá ser indicada explicitamente a linha de CS (*Chip Selection*) da memória resultante. Cada endereço de memória deverá armazenar 8 bits. **(2 Val)**



- 3** Considere uma máquina com um espaço de endereçamento de memória RAM de 36 bits de endereço, uma cache com capacidade de armazenamento de dados de 64 Kbytes, mapeamento direto e blocos de 1 byte **(2,5 Val)**

- a) Quantos bits são necessários para a *tag*? Justifique.
- b) Qual a capacidade total desta cache, contando com os bits da *tag* mais os *valid bits*?

Parte Prática

1. Desenvolva um programa em linguagem Assembly 8086 que dado um qualquer vetor de 10 números de 8 bits (sem sinal), **Vetor1**, construa um segundo vetor, **Vetor2**, com a posição em **vetor1** dos números primos existentes em **Vetor1**. **(2,5 Val)**

Exemplo

Vetor1:
29, 2, 6, 10, 12, 25, 7, 11, 100, 23

Vetor2:
0, 1, 6, 7, 9

2. Elabore o **procedimento** em *Assembly* 8086, de nome **getData**, que copie os caracteres existentes numa área específica do ecrã (quadrado), e distribua esses caracteres de acordo com o seu tipo, dígito ou outro carater.

O procedimento recebe a posição do canto superior esquerdo da área (linha e coluna), através dos registos AL e AH, a dimensão de cada lado do quadrado, no registo CL, e em SI e DI, são passados os endereços das variáveis **dígitos** e **outros**. Estas variáveis encontram-se definidas como array de bytes, com espaço suficiente para armazenar os dados pretendidos.

O procedimento deverá ser implementado com recurso **ao acesso direto à memória de vídeo, não se aceitando soluções que recorram a interrupções**. Considere que os parâmetros de entrada estão bem definidos e a área não excede a dimensão do ecrã. Além disso, o carater **espaço** que possa existir nas células, não deverá ser copiado. Apenas deve implementar o procedimento **getData**.

Veja o exemplo de utilização do procedimento apresentado em baixo (o resultado encontra-se a negrito).

	0	1	2	3	4	5	6	7	8	9	10	...
0												
1												
2		T	A	C		1	º	s	e	m		
3		1	a	n	o							
4												
5		D	E	I								
6												
7												
8												
9												
10											2	0
11											2	4
...												

```
mov AL, 2
mov AH, 1
mov CL, 9
lea SI,dígitos
lea SI,outros
call getData

/*Resultado será:
  Dígitos=1,1,2,0,2,4
  Outros=T,A,C,º,s,e,m,a,n,o,D,E,I
*/
```

(2,5 Val)

```
.8086
.model small
.stack 2048
DATA_HERE SEGMENT
...
DATA_HERE ENDS
CODE_HERE SEGMENT
    ASSUME CS:CODE_HERE, DS:DATA_HERE
START:mov ax, DATA_HERE
    mov ds, ax
    mov bx,0b800h
    mov es,bx
    ...
    mov ah,4ch
    int 21h
CODE_HERE ENDS
END START
```

BOA SORTE! ☺